

bot-swarm: a distributed swarm for cooperative maze solving

Andrew Chang-DeWitt
CS 451 - Parallel & Distributed Systems
Illinois Institute of Technology
Spring 2026

Introduction

`maze-robot` [1] is a teaching toy for young children to learn programming without screens. Instead of writing text code, children build programs using physical puzzle pieces; the robot scans them, compiles the result to machine code, & executes it to explore a maze. As part of building that project, I wrote a text-based simulation environment — a Rust library of traits & types that lets a simulated robot navigate text-encoded mazes. `bot-swarm` extends that simulation with distributed swarm capabilities: instead of one robot solving a maze alone, a swarm of robots running across multiple network nodes can now explore the same maze in parallel.

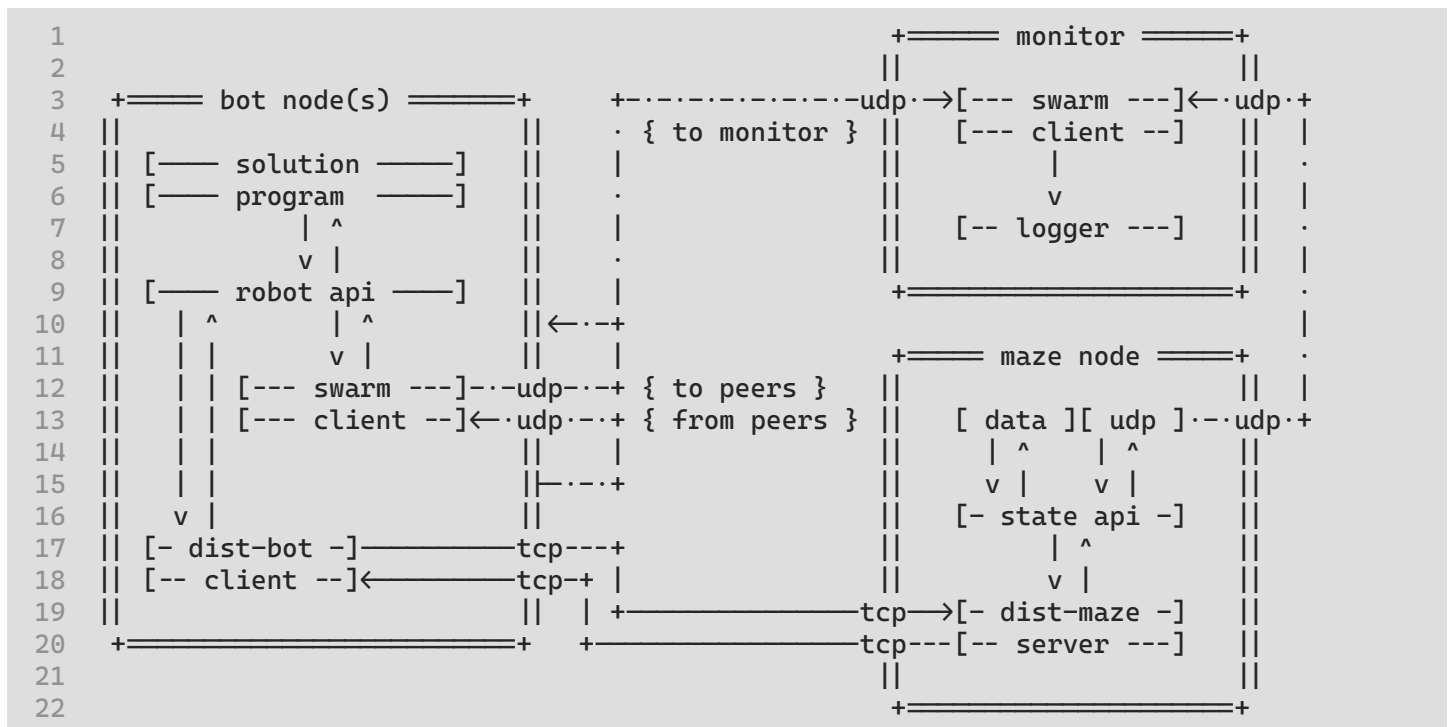
The implementation has two layers. The first is an in-process multi-robot layer (`MultiTextMaze` , `MultiTextRobot`) that simulates a swarm sharing a single maze in memory — useful for testing & single-machine demos. The second is a network layer where each robot runs as a separate process on its own node, with maze state held on a dedicated server & inter-robot coordination happening over UDP broadcast. Both layers share the same solver logic, which is possible because of a trait-based abstraction that hides transport details from the algorithm.

What it does

Architecture

The distributed system is organized around three node roles:

1. **Maze node** — a single server holding authoritative maze state. Bot nodes query it over TCP any time they want to look at or move through a cell, ensuring consistent state across all robots. The server uses a thread pool to handle concurrent connections, following the pattern from *The Rust Programming Language* [2].
2. **Bot nodes** — N separate processes, each running a `DistRobot` that connects to the maze node over TCP & joins a UDP broadcast group (`Swarm`) for peer-to-peer coordination.
3. **Monitor node** — an optional observer designed to receive log messages from all other nodes. This role is defined in the design but not yet implemented.



Trait abstraction

The library is built around three core traits. `Maze` exposes `look_dir(dir)` & `move_dir(dir)` — the minimal interface for navigating a cell-based environment. `Robot` wraps a `Maze` & exposes `peek(dir)` & `go(dir)` to callers. A key invariant is that `go` takes `&self` rather than `&mut self`, so robots can be shared across closures without wrapping them in a mutex. This is achieved through `RobotInternal`, which holds the underlying `Maze` in a `RefCell<Box<dyn Maze>>` and handles interior mutability. `MultiMaze` extends `Maze` with multi-robot support, adding `add_bot()`, `has_bot(id)`, `bot_ids()`, & per-bot variants of `look_dir` & `move_dir`.

Because `TextMaze`, `DistMazeClient`, & `MultiTextMaze` all implement these traits, the example DFS solver in `examples/dist_bot.rs` is structurally identical to the one in `examples/text_dfs.rs` — the transport layer is swapped out underneath without touching the algorithm.

This ability to apply the same solution across different Maze implementations will allow users to develop solutions in one of any supported environment that fits their process without worrying about the underlying implementations. For example, a user could choose to develop most of a solution on a `TextMaze` or `DistMazeServer`, then deploy it later to actual robots when those Maze implementations are supported by the library.

How to use it

For the simulation, Mazes are encoded as plain text files. Four characters have special meaning:

- `S` : robot start position
- `F` : finish
- `+` : wall
- (space): open space

A trivial example:

```
1  ++++++
2  +   +
3  S + F
4  ++++++
```

These characters are interpreted as maze Cells, which together form a grid that describes a Maze.

A User can program a `Robot` to solve such mazes using the core API:

- `Robot::look_dir(Direction) → Result<Cell, MazeError>`, which can inspect any of the 4 cells directly neighboring the one the robot is currently occupying.
- `Robot::move_dir(Direction) → Result<(), MazeError>`, which has the robot attempt to move one cell in the given direction.

For example, a hardcoded solution to the above maze might look like this:

```
1  use maze_robot::{TextRobot, TextMaze};
2
3  fn main() {
4      let maze = ... // above example as a &str
5      let robot = TextRobot::try_from(maze)
6          .expect("robot should initialize");
7
8      Robot.go(Direction::East).unwrap();
9      Robot.go(Direction::North).unwrap();
10     Robot.go(Direction::East).unwrap();
11     Robot.go(Direction::East).unwrap();
12     Robot.go(Direction::South).unwrap();
13     Robot.go(Direction::West).unwrap();
14 }
```

Some more in-depth examples can be found in the `./maze-robot/examples` directory of the source code [1]. The following instructions discuss how to run these examples.

Single-robot text maze

The simplest entry point is the `text-dfs` example, which runs one robot through a text-encoded maze file & prints the explored cells:

```
1 cargo run --example text-dfs -- examples/test-maze.txt
```

Visited cells are marked `•` in the output; walls are marked `▩`. Tests are embedded in the example file & can be run with:

```
1 cargo test --example text-dfs
```

Distributed maze (local)

To run a distributed swarm locally, start the maze node in one terminal & one or more bot nodes in separate terminals. The `--local` flag tells bots to broadcast on `127.255.255.255` (loopback subnet) instead of `255.255.255.255`, which is required when all processes share a host:

```
1 # Terminal 1 - maze node
2 cargo run --example dist-maze -- examples/test-maze.txt 5000
3
4 # Terminal 2 - first bot
5 cargo run --example dist-bot -- 127.0.0.1:5000 5001 --local
6
7 # Terminal 3 - second bot (optional; same swarm port)
8 cargo run --example dist-bot -- 127.0.0.1:5000 5001 --local
```

The first argument to `dist-bot` is the maze node's address; the second is the swarm broadcast port. All bots that share a port form a swarm. Each bot filters out its own UDP broadcasts using a nonce set at startup.

Distributed maze (on FABRIC)

On FABRIC, each process runs on a separate node on the same slice network. Drop the `--local` flag so bots use the full subnet broadcast address; replace the maze node address with its actual slice IP:

```
1 # On the maze node
2 cargo run --example dist-maze -- examples/test-maze.txt 5000
3
4 # On each bot node
5 cargo run --example dist-bot -- 10.0.0.1:5000 5001
```

Because bots communicate via UDP broadcast, all nodes must be on the same L2 subnet — FABRIC slices satisfy this by default. Running three or more bot nodes on separate FABRIC nodes provides the intended distributed execution model.

Library API

For building custom solvers, the public API is:

```
1 // Single robot (text maze)
2 let robot: TextRobot = new_bot(maze_str)?;
```

```

3 let cell = robot.peek(Direction::East)?; // look without moving
4 robot.go(Direction::North)?;           // move one cell
5
6 // Distributed robot (network maze)
7 let robot = DistRobot::try_build("10.0.0.1:5000".parse())?;
8 let robot = robot.join_swarm(5001)?;
9 robot.peek(Direction::East)?;
10 robot.try_send(msg)?; // broadcast to swarm peers
11 robot.try_rcv:<MyMsg, 32>()?; // non-blocking receive

```

The `Cell` type returned by `peek` has four variants: `Open`, `Wall`, `Finish`, & `Occupied(u32)` (another robot is present). `go` returns `Err(MazeError`

)` if the target cell is a wall or is occupied.

Limitations

No shared coordinate frame

The most significant limitation is that bots have no shared frame of reference. Each `DistRobot` tracks its position relative to its own starting cell: the bot that starts at `S` considers that cell `(0, 0)`, & a peer that also starts at `S` has its own independent `(0, 0)`. When a bot broadcasts "I've visited `(3, 2)`", its peers have no way to translate that coordinate into their own navigation. Peer observations are stored & printed in the solution summary, but they cannot guide the receiving bot's search decisions. The swarm's shared information is observational rather than actionable.

Exploration is not coordinated

Related to the coordinate frame problem: bots don't partition the maze or avoid duplicating each other's work. Each runs an independent DFS from the same start cell. In the worst case — when all bots take the same path — N bots provide no parallelism benefit over one. In practice they diverge somewhat when one bot's position causes another to receive an `Occupied` response & backtrack, but this is an accidental side effect rather than an intentional coordination strategy. A proper implementation would need to either share a coordinate frame or use a different exploration algorithm designed for multi-agent settings.

Monitor node not implemented

The design includes a third node role: a monitor that receives structured log messages from all other nodes & echoes them to stdout for runtime observability. The UDP infrastructure for broadcasting to a monitor address exists in the codebase, but the monitor process itself is not implemented. Currently, each node logs only to its own stdout.

Secondary limitations

- **UDP is subnet-local.** Swarm broadcast uses `255.255.255.255` (or `127.255.255.255` locally), which does not route across subnets. This works on FABRIC slices but rules out deployment over broader networks without replacing the transport.
- **Fixed message sizes.** Cell responses are encoded as `[u8; 5]` & swarm messages as `[u8; 32]`. There is no framing or variable-length support, which limits the amount of information any single message can carry.
- **No fault tolerance.** If the maze node crashes, all bot nodes fail with a connection error; there is no replica, reconnect logic, or graceful degradation. Similarly, if a bot node exits early, its peers receive no notification.
- **No authentication or encryption.** TCP & UDP channels are unencrypted & unauthenticated. Any host on the same subnet can inject valid-looking messages into the swarm.

- **DFS is not optimal.** The solver finds *a* path to the finish, not the shortest one. Because multiple bots may explore in parallel & halt on the first finish found, the reported path length depends on which bot happens to find the finish first.
- **Visual output is limited to 62 bots.** Bot IDs in the rendered maze output use `0-9`, `A-Z`, & `a-z`; more than 62 bots cannot be visually distinguished.

What I'd add

The two most significant limitations — the missing monitor node & the lack of a shared coordinate frame — would be my first targets.

The monitor node is the simpler of the two. The UDP infrastructure for broadcasting to a monitor address already exists; what's missing is a process that listens on that port & does something with what it receives. I'd implement a monitor that periodically renders each bot's in-progress solution as an ASCII string — using the same cell-marking scheme as `text-dfs` — & prints it to stdout, so an operator can watch the swarm explore in real time rather than waiting for a bot to reach the finish.

The more interesting addition would be a coordinate system reconciliation utility in the library. Right now, when a bot receives a ``Cell`

(id) response, it knows another robot is in an adjacent cell but can't do anything useful with that information. The robot's ID & the direction of the occupied cell together give a known spatial relationship between the two bots' positions: bot A is one cell north of bot B, which means A's `(0, 0)` & B's `(0, 0)` can be related by a fixed offset. I'd expose a utility that tracks these observed adjacencies & uses them to build a mapping between peer coordinate systems as bots encounter each other during exploration. Once two bots' frames are reconciled, cells seen by one can be translated into the other's frame & acted on directly.

That reconciliation utility would unlock two things. First, bots could maintain a shared pool of cells that have been *seen* but not yet *visited* — discovered by any peer whose coordinate frame is known — & draw from it to avoid duplicating exploration. This would be a meaningful improvement over the current situation where bots independently re-explore the same paths. Second, the monitor node could use the same mapping to render a unified in-progress solution that composites what all bots have seen, distinguishing between regions explored jointly & regions only one bot has reached.

As a stretch goal, I'd implement a distributed queue data structure backed by a conflict-free replicated data type (CRDT) to manage the pool of unvisited cells. A grow-only set CRDT is a natural fit: bots add cells to the set as they see them & mark cells as claimed when they choose to visit them, & the set converges to a consistent state across all replicas regardless of message ordering or delivery. The `Swarm` UDP broadcast layer already provides the transport; the CRDT would be the data structure sitting on top of it, replacing the ad-hoc broadcast messages the current implementation uses for coordination.

References

- [1] Andrew Chang-DeWitt. 2026. maze-robot. GitHub. Retrieved May 12, 2026 from <https://github.com/andrew-chang-dewitt/maze-robot/tree/swarm/main>.
- [2] Steve Klabnik, Carol Nichols. 2023. The Rust Programming Language. Building a Multithreaded Web Server. *In The Rust Programming Language*. No Starch Press, undefined. <https://doc.rust-lang.org/book/ch20-00-final-project-a-web-server.html>